

AI CODE IN-THE-WILD SECURITY RISK RESEARCH REPORT

AI 生成代码在野安全风险研究报告

In-the-Wild Analysis & Vulnerability Assessment
真实场景分析与漏洞评估

报告范围 • REPORT SCOPE //

AI 代码生成 • 安全风险 • 趋势洞察 • 漏洞分析

AI Code Generation & Security Risks Trends & Vulnerability Analysis

日期 • DATE //

2025 年 12 月

December 2025

联合发布 • JOINTLY RELEASED BY //



腾讯安全平台部
悟空代码安全团队



北京大学
Narwhal-Lab



复旦大学
系统软件与安全实验室
威胁情报小组

Abstract

With the AI code generation technology driven by Large Language Models (LLMs) being increasingly integrated into the entire software development lifecycle, it not only improves R&D efficiency, but also reshapes the security boundaries of the software supply chain.

This report systematically evaluates the current capability status, potential risks, and evolutionary trends of the existing AI code generation technology. We have constructed a hierarchical capability system ranging from IDE-assisted tools to autonomous AI software engineers, and analyzed the vulnerability injection risks, data leakage hazards, and compliance challenges caused by code hallucinations, knowledge truncation, and training data bias.

Based on the empirical research on the open-source ecosystem, we further reveal the dual role of AI in the vulnerability lifecycle: it can not only improve the efficiency of vulnerability remediation, but also may become a source of new defects.

The study indicates that with the increase in the proportion of AI-generated code, the role of developers is shifting from "code writers" to "reviewers and validators".

Finally, this report proposes the construction of a full-lifecycle security governance framework to ensure the realization of secure and reliable AI-aided development in the continuous integration environment.

Key Words: AI Code Generation; Software Supply Chain Security; Code Hallucination; Vulnerability Analysis; Code Security Governance

Contents

1	Overview	4
2	Capabilities and Application Overview of AI Code Generation	5
2.1	Classification of Core Capabilities	5
2.2	Mainstream AI Code Generation Products and Platforms	6
2.3	Summary	7
3	Risks and Challenges of AI-Generated Code	8
3.1	Technical Characteristics and Inherent Contradictions	8
3.2	Direct Security Risks	9
3.3	Indirect and Compliance Risks	9
3.4	Summary	10
4	Trends and Characteristics of AI-Generated Code	11
4.1	Temporal Development Trends	11
4.2	Distribution of Programming Languages	13
4.3	Summary	15
5	Security Risks and Vulnerability Characteristics of AI-Generated Code	16
5.1	Evolution of AI's Role in the Vulnerability Lifecycle	16
5.2	Characteristic Profile of AI-Introduced Vulnerabilities	17
5.3	Summary	19
6	Risk Mitigation Recommendations	20
6.1	Establish Multi-dimensional Evaluation Benchmarks	20
6.2	Enhance Model Intrinsic Security	21
6.3	Human-Machine Collaborative Governance	21
6.4	Summary	21
7	Conclusion	23

1 Overview

With the leap-forward development of Large Language Model (LLM) technology, Artificial Intelligence (AI) has been deeply integrated into all links of software development, among which AI code generation technology is triggering a profound paradigm revolution. From IDE-integrated tools such as GitHub Copilot and Integrated Development Environments (IDEs) such as Cursor, to emerging autonomous AI software engineers such as Devin, these tools have greatly improved development efficiency and innovation speed through automating code writing, debugging, testing, and even project deployment.

However, this technological innovation is a double-edged sword. While enjoying the convenience brought by automation, an unprecedented and increasingly severe security challenge has emerged: whether AI-generated code is safe and reliable.

When AI becomes an emerging, automated code contributor in the software supply chain, its inherent technical characteristics, biases in training data, and interaction patterns with developers may introduce novel risks that have not been fully considered by traditional software security systems. These risks are not limited to traditional vulnerability injection, but also extend to multiple dimensions including software supply chain security, intellectual property compliance, and even the security culture of development teams. Therefore, conducting a systematic and in-depth analysis of the security impacts of AI code generation technology has become an urgent and critical issue for both academia and industry.

This report focuses on the security threat landscape of AI code generation technology. First, the report defines the capability boundaries and typical application modes of this technology, establishing a benchmark for risk assessment. Subsequently, it analyzes the characteristics of code from the perspective of generation mechanism, and elaborates on the multi-dimensional security risks derived therefrom. Through quantitative analysis of the penetration trend of AI-generated code in the open-source ecosystem and its evolution law in the vulnerability lifecycle, this report aims to provide data support for understanding the actual security effectiveness of AI code generation.

2 Capabilities and Application Overview of AI Code Generation

2.1 Classification of Core Capabilities

AI-driven code generation technology, especially Large Language Models (LLMs) built on the Transformer architecture, has evolved from a simple code completion tool into an intelligent ecosystem covering the entire software development lifecycle. To systematically evaluate its impacts on security and efficiency, we categorize the existing technical capabilities into three main levels based on their scope of action and abstraction level in the software development lifecycle: micro-level assistance, macro-level autonomous construction, and domain-specific generation.

At the micro level, the most widely adopted application form of AI is the real-time assistance function deeply integrated into Integrated Development Environments (IDEs). Such capabilities transform traditional development tools into proactive programming collaborators, with intelligent code completion as the core, which realizes a paradigm shift from syntax-level prompts to semantic-level generation, and can generate code blocks based on the global context of the project. Code interpretation and document generation are designed to bridge the gap between natural language and programming languages, improving the maintainability of software. Meanwhile, in terms of real-time debugging and error fixing, AI models can conduct static analysis during the coding phase, proactively identify potential defects and provide repair suggestions for errors. This "real-time feedback" mechanism effectively promotes the implementation of the Security Shift Left development concept.

At the macro level, with the rise of AI Agent technology, the role of AI is shifting from an assistant to a system designer and executor. This level goes beyond simple code snippet generation, and has the capabilities of project-level scaffolding construction and full-process automation. For example, Devin and the open-source project OpenHands demonstrate the potential of AI in long-term planning and execution: they can not only automatically generate configuration files and infrastructure required for projects based on natural language descriptions, but also independently complete the closed loop from requirement analysis, code writing to error debugging. The core prototype and function generation capabilities support the direct transformation of abstract business logic into runnable code covering front-end interaction, back-end logic and data persistence layer, which greatly accelerates the iteration cycle from proof of concept (PoC) to minimum viable product (MVP).

In addition to general-purpose programming languages, AI also performs well in handling Domain-Specific Languages (DSLs). Among these applications, database query generation is particularly mature: AI models with schema-aware capabilities can transform complex natural language data retrieval requirements into accurate and efficient SQL query statements. Similarly, in the field of system integration and DevOps (Development and Operations), AI can generate API calling code complying with specific specifications or functionally correct command-line scripts according to requirements such as generating management instructions for Kubernetes, which significantly improves the operation and maintenance efficiency in cloud-native environments.

2.2 Mainstream AI Code Generation Products and Platforms

With the rapid advancement of underlying Large Language Model (LLM) technologies, AI code generation has evolved into a prosperous and diverse technological ecosystem. Relevant products and platforms on the market are distinctive in form, function, and integration depth, jointly shaping the current technological landscape. These products can be categorized into four mainstream paradigms: IDE/Editor Integrated Plugins, Conversational AI, Integrated Development Platform, and the emerging AI Software Engineer Agents, as shown in Table 1.

Table 1: Classification and Representative Products of Mainstream AI Code Generation Tools

Classification Paradigm	Representative Products and Platforms
IDE/Editor Integrated Plugins	GitHub Copilot, CodeBuddy, Tabnine, Amazon CodeWhisperer, JetBrains AI Assistant, Tongyi Lingma, etc
Conversational AI for Code Generation	OpenAI ChatGPT, Google Gemini, Anthropic Claude, Grok, Phind, Perplexity AI, etc
Integrated Development Platforms	Cursor, Codex, Claude Code, GitHub Codespaces, Qoder, Trae, Antigravity, etc
AI Software Engineer Agents	Devin, Devika, OpenHands, Aider, Sweep

IDE/Editor Integrated Plugins are currently the most mature and widely adopted form of tools. Such products are embedded into existing development environments in the form of extension plugins, providing fine-grained code completion and suggestions without disrupting the developers' flow. Typical representatives of this paradigm include GitHub Copilot, CodeBuddy, Tabnine (supporting private model fine-tuning), and Amazon CodeWhisperer (with built-in security scanning capability).

Conversational AI platforms (such as ChatGPT, Gemini, and Claude) support higher-level programming assistance through natural language interfaces, including problem analysis, solution deduction, and multi-step task decomposition, and have become important technical consultation portals for developers. Integrated Development Platforms further enhance this capability: by deeply integrating AI into the complete development processes of coding, debugging, refactoring, running, and collaboration, they deliver an AI-native software engineering experience. Currently representative practices include Cursor, Claude Code, and GitHub Codespaces.

AI Software Engineer Agents represent the cutting-edge exploration trend in this field, with the goal of evolving from tool-based assistance to task-autonomous agents. Systems represented by Devin released by Cognition AI are capable of completing end-to-end processes such as task planning, code implementation, debugging and verification, and even simple deployment, and have spawned a variety of open-source or lightweight alternatives such as OpenHands, Devika, Aider, and Sweep. This direction indicates that the software engineering paradigm may undergo further automated and autonomous transformation.

2.3 Summary

This chapter systematically organizes the current AI code generation technology from two dimensions: capability structure and product ecosystem. AI has been equipped with a variety of functions, ranging from local code completion, interpretation and repair, to project-level scaffolding construction and automated development, and further to domain-specific generation for database and operation and maintenance scenarios, gradually covering the main links in the software development lifecycle. These capabilities are implemented in different forms including IDE plugins, conversational interfaces, integrated development environments, and AI Software Engineer Agents, forming a multi-level technical system spanning from lightweight integration to high autonomy.

AI code generation is evolving from auxiliary tools to development entities with autonomous execution capabilities, and product forms have begun to show trends of stratification and specialization. This evolution has significantly improved R&D efficiency, but also introduced new issues regarding security, reliability, and process controllability. The analysis in this chapter lays the necessary technical foundation and ecological background for subsequent risk assessment and detection scheme design.

3 Risks and Challenges of AI-Generated Code

While reshaping the software development process, AI Code Generation Technology has also introduced a series of unique intrinsic characteristics and multi-dimensional associated challenges. A thorough analysis of these characteristics is a prerequisite for evaluating their profound impacts on software security, code quality, and development culture. These challenges are not isolated technical flaws, but systematic issues that run through model training, human-computer interaction, legal compliance, and developer behavior patterns. Figure 1 presents the risk classification framework spanning technical, legal, and organizational dimensions.

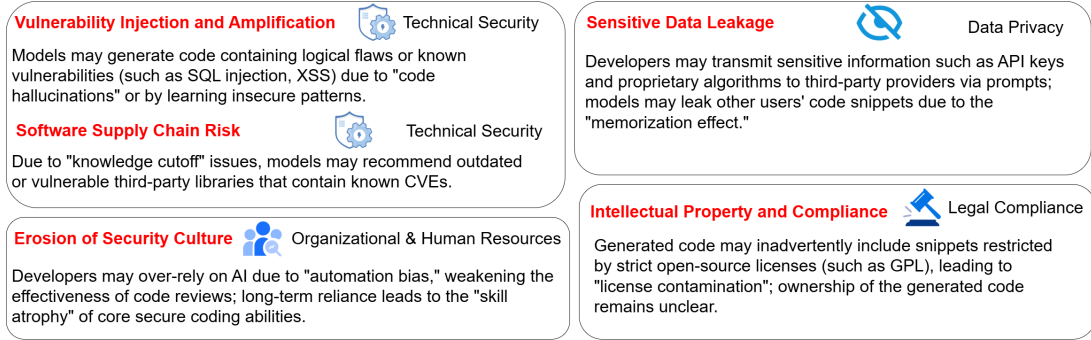


Figure 1: Main Risk Classification of AI Code Generation

3.1 Technical Characteristics and Inherent Contradictions

The core advantage of AI code generation technology lies in significantly improving the efficiency of the software development lifecycle by automatically generating boilerplate code, algorithm implementations, and test cases. Its effectiveness is rooted in advanced context-aware capabilities – modern AI models can comprehensively analyze Abstract Syntax Trees (ASTs), project-level dependencies, and even code patterns across entire libraries, thus evolving from simple code completion tools into programming assistance systems with logical reasoning capabilities.

However, this capability exhibits significant inherent contradictions:

- Conflict between Standardization and Fragmentation:** Models fine-tuned on specific codebases can serve as powerful tools for enforcing coding standards; conversely, uncustomized general-purpose models, due to the heterogeneity of their training data, may introduce inconsistent coding styles and architectural patterns into the project, thereby undermining the long-term maintainability of the codebase.
- Prompt Sensitivity:** The output quality is highly positively correlated with the quality of input prompts. Vague or ambiguous natural language descriptions can easily cause the model to generate code with deviated or even incorrect functions. This requires developers to master precise requirement analysis and logical abstraction capabilities (namely Prompt Engineering). If the competence of the team in this aspect is uneven, the cost of repeatedly debugging poor-quality code may offset the efficiency dividends brought by automation.

3.2 Direct Security Risks

Despite its powerful capabilities, one of the most severe current challenges of AI code generation technology is its inherent reliability issues, which directly translate into a variety of security risks. The primary risk is vulnerability injection caused by Code Hallucination. This phenomenon refers to the model generating code that is syntactically correct but has severe logical flaws in semantics with an extremely high "degree of confidence", which easily triggers developers' Automation Bias. These flaws often directly map to serious security vulnerabilities.

Case Analysis: Existing studies have found that when AI tools (such as early versions of GitHub Copilot) are required to implement encryption functions, the model may prioritize generating code that uses the deprecated DES algorithm or ECB operating mode – which have serious security flaws – because these may appear more frequently in the training data than the more secure and modern AES-GCM mode.

Another major direct risk stems from the model's Knowledge Cutoff. This characteristic causes the model to be unaware of the latest security announcements, and it may recommend the use of third-party libraries known to contain severe Common Vulnerabilities and Exposures (CVEs) in the code it generates.

Case Analysis: After the disclosure of critical vulnerabilities such as Log4j's "Log4Shell" (CVE-2021-44228) or OpenSSL's "Heartbleed", a model with an unupdated knowledge base will continue to generate configurations and code that use vulnerable version dependencies, which is equivalent to actively introducing known attack vectors into the software supply chain. The interaction process with AI services itself also constitutes a data leakage risk. Any sensitive information included by developers in prompts, such as API keys or proprietary algorithms, may be transmitted and stored on third-party servers. The model's Memory Effect may also cause it to inadvertently reproduce unique code snippets or sensitive information belonging to other users in the training data when generating code, resulting in cross-user data leakage.

3.3 Indirect and Compliance Risks

The ambiguity of ownership of AI-generated code brings challenges to IP management and open source compliance. The training data of the model comes from millions of open-source projects that follow different licenses. If the model generates a piece of code that essentially originates from a strong copyleft license such as GPL, and the developer uses it in a commercial closed-source project, it may trigger License Contamination, bringing huge legal and commercial risks to the enterprise.

Case Analysis: This risk is not purely theoretical. One of the core disputes in the class-action lawsuit against GitHub Copilot (J. Doe 1 v. GitHub, Inc.) is whether the tool reproduces code snippets protected by strict open-source licenses without authorization and attribution, thereby posing potential compliance risks to downstream users. The ambiguity of code ownership may also constitute an obstacle in corporate mergers and acquisitions or technical due diligence. In addition to the legal and compliance aspects, the widespread application of AI tools may also have long-term, more hidden negative impacts on the security culture of development teams. Developers' Automation Bias may lead them to blindly trust the results generated by AI, thereby significantly reducing the depth and breadth of code review and weakening the role of this key security assurance link. Long-term reliance on AI to complete standard secure coding tasks (such as input validation) may lead to the "degradation" of the team's overall core security skills. If

developers gradually lose the ability to independently identify and fix complex security issues, the organization will form a dangerous path dependence on AI tools, fundamentally weakening its security defense-in-depth capabilities and technical resilience.

3.4 Summary

This chapter systematically analyzes the duality of AI code generation technology. While improving efficiency, it is accompanied by active vulnerability injection risks caused by Code Hallucination and Knowledge Cutoff, as well as data leakage risks in the interaction process. The more far-reaching indirect risks involve intellectual property compliance and the degradation of developers' core security skills. These multi-dimensional risk factors jointly reveal the core dilemma faced by AI-assisted programming: while the degree of automation is improving, security controllability is facing unprecedented challenges.

4 Trends and Characteristics of AI-Generated Code

4.1 Temporal Development Trends

Looking at the implementation process of AI code generation technology in the open-source community, we have tracked and found that this is not a simple linear penetration process, but a complex evolution full of twists and turns, self-correction, and paradigm restructuring (Figure 2). Contrary to early optimistic predictions that AI would quickly and seamlessly take over software development, the interactive relationship between AI and human developers has experienced a dialectical development stage from Tool Worship to Trust Crisis, and finally to Rational Symbiosis. This evolution trajectory not only reflects the iteration of model capabilities, but also profoundly reflects the adaptability and reshaping ability of the software engineering industry when facing disruptive technologies.

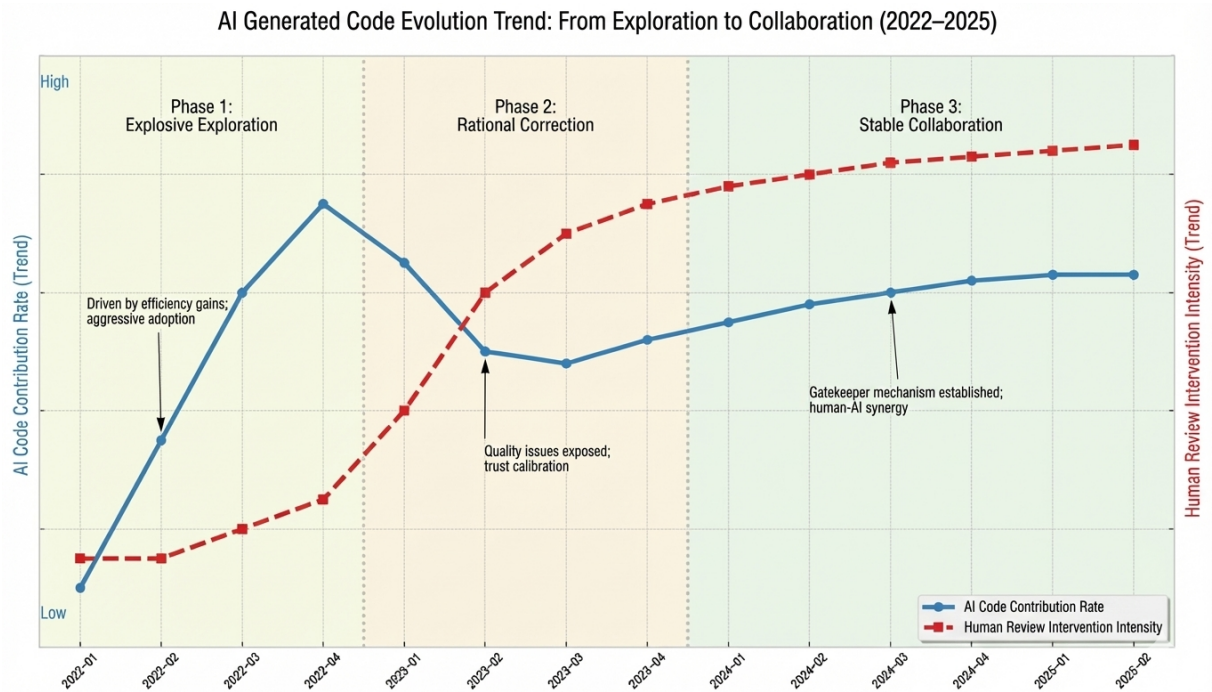


Figure 2: Trends in the Use of AI in Code Generation

During the initial period when AI-assisted programming tools entered the market on a large scale, the open-source community experienced a notable Explosive Exploration Period. The core feature of this phase was the radical adoption of a breadth-first approach. Benefiting from the seamless integration of IDE plugins and extremely low barriers to use, AI technology quickly crossed the chasm of early adopters, achieving astonishing growth in penetration rate in a very short time. Driven by the efficiency dividends brought by the new technology, developers at this time often held an exploratory or even curious mindset, and tended to widely try using AI-generated content in various programming tasks. Whether it was simple function completion, boilerplate code generation, or complex algorithm implementation, AI suggestions were often accepted and submitted without discrimination. Data flow trends showed that the proportion of AI-generated code in the total amount of newly added code presented a steep upward curve, reflecting the industry's extremely high expectations and enthusiasm for the concept of "automated programming" – AI was regarded as a silver bullet that could improve productivity immediately.

However, with the deepening of application, this early fanaticism inevitably encountered the resistance of real-world engineering complexity, and the industry subsequently entered a long and painful Period of Rational Return and Trust Calibration. In this phase, the empirical data curve showed obvious oscillations, stagnation, or even a decline. This was not a regression of technology, but the maturation of industry cognition. With the expansion of codebase scale, the limitations of AI-generated code in long-context understanding, misunderstandings of domain-specific business logic, and potential non-functional defects began to be exposed intensively. Developers gradually realized that although AI could generate hundreds of lines of code in a few seconds, the subsequent debugging, review, and maintenance costs might far exceed expectations. The emergence of this technical debt led to the fading of automation bias: developers shifted from blind adoption to prudent evaluation, and consciously narrowed the scope of AI use. Data indicated that during this period, the intervention rate of AI in high-risk, high-complexity, or core architecture-related scenarios saw a structural pullback, which marked that developers were re-establishing the dominant position of humans in code quality control.

In the recent phase, we have observed that the evolution of AI-generated code has passed through the trough of disappointment and entered a brand-new Period of Stable Collaboration and Ecological Integration. Although the total amount of AI-generated code has rebounded and tended to saturate after adjustments, the production relationship behind it has undergone a qualitative transformation. At this point, AI is no longer regarded as an "independent contractor" trying to replace developers, but is redefined as a high-throughput intelligent assistant. Empirical analysis reveals a key niche change: the role weight of human developers in the human-machine loop has undergone a fundamental shift – from a mere code writer to a high-level reviewer and architect.

Under this mature collaboration paradigm, the division of labor between AI and humans has become clearer than ever: AI is assigned to handle tasks that are highly repetitive, highly patterned, and easy to verify, such as unit test writing, document generation, interface adaptation, and the implementation of basic glue code; Humans, on the other hand, concentrate their energy on areas that are difficult for AI to reach, such as complex system design, clarification of ambiguous requirements, decision-making on core business logic, and final security checks.

It is particularly worth noting that data trends also reveal the rise of AI in the field of self-repair: more and more code changes are no longer just the generation of new functions, but the optimization and defect repair of existing code by AI. This trend indicates that AI is permeating from the "upstream" of software development to the "full lifecycle", taking charge not only of "creating" code, but also of "maintaining" it.

To sum up, the evolution of AI code generation technology in the open-source world is a history from "unrestrained growth" to "meticulous cultivation". Every fluctuation on the timeline represents a precise mapping of the industry's understanding of the boundary of AI capabilities. The current dynamic balance of "AI is responsible for high-efficiency output, while humans are responsible for high-standard checks" may be the standard normal state in the field of software engineering for a long time in the future. This shows that technological progress has not simply eliminated humans, but has forced us to evolve more advanced discrimination capabilities and systematic thinking to control this powerful tide of automation.

4.2 Distribution of Programming Languages

The penetration level of AI code generation technology is not evenly distributed across all tech stacks, but shows significant hierarchical characteristics (Figure 3). This distribution pattern is mainly driven by two core factors: the richness of open-source training corpora, and the syntactic redundancy of the language itself.

At the macro level, high-resource languages with massive open-source ecosystems such as Python, JavaScript, and TypeScript occupy an absolutely dominant position in AI-generated code. These languages feature flexible syntax and high fault tolerance, enabling AI to easily generate large sections of business logic. They are closely followed by enterprise-level languages such as Java and Go. Although their logic is rigorous, due to the existence of a large amount of boilerplate code, developers tend to use AI to eliminate repetitive work. In contrast, for system-level languages such as Rust and C++, due to strict restrictions on memory safety and lifecycle, the adoption rate of AI-generated code is relatively low, and developers often maintain a more prudent human-machine collaboration distance in these scenarios.

This macro-level penetration distribution provides a context for our understanding of security risks: high penetration often implies a potential high-risk exposure surface. However, when we shift our perspective from the total generation volume to the specific high-risk scenario of vulnerability remediation, the situation changes subtly.

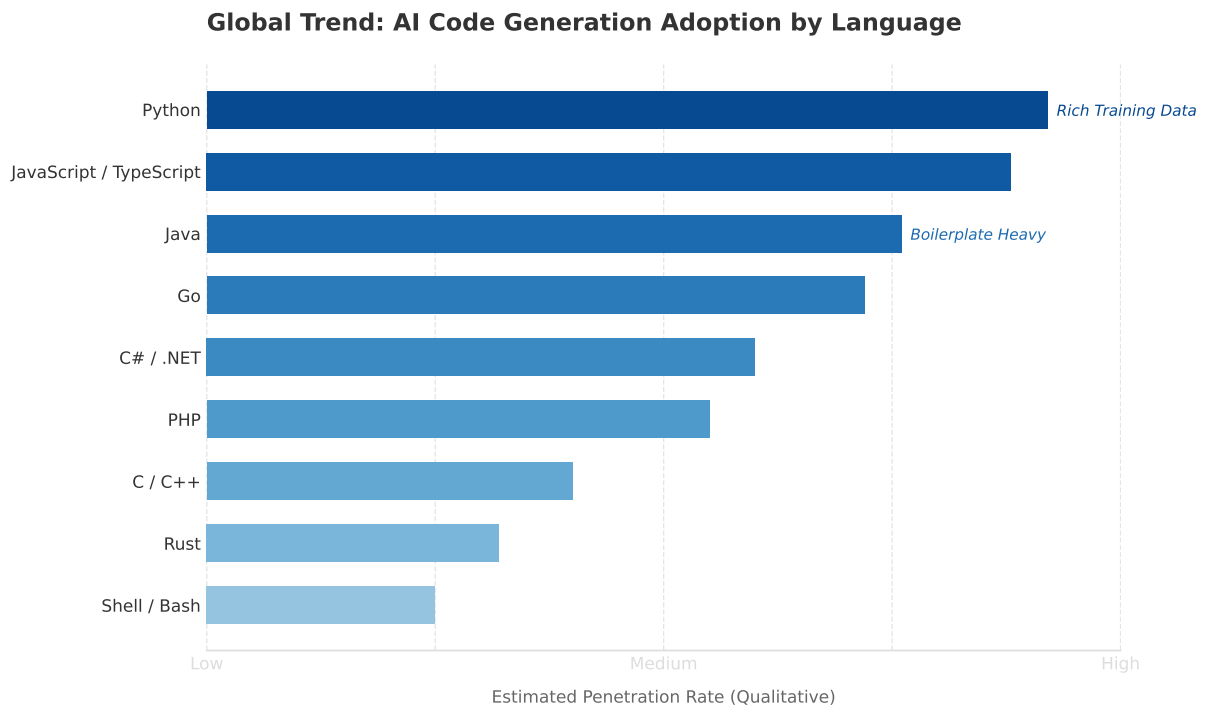


Figure 3: Relative Proportions of AI-Generated Code in Different Programming Languages

Analysis of multiple programming languages and configuration file types (Figure 4) shows that after vulnerability remediation, the AI code ratio of most languages presents a general upward trend. This phenomenon is consistent with the repository-level observations, further confirming the growing influence of AI-assisted programming tools in the software maintenance lifecycle.

Whether in low-level system languages, general-purpose high-level languages, or front-end scripting and markup languages, the proportion of AI-generated code has generally increased. This indicates that developers widely leverage AI tools to accelerate the development process when addressing code defects and iterating functions.

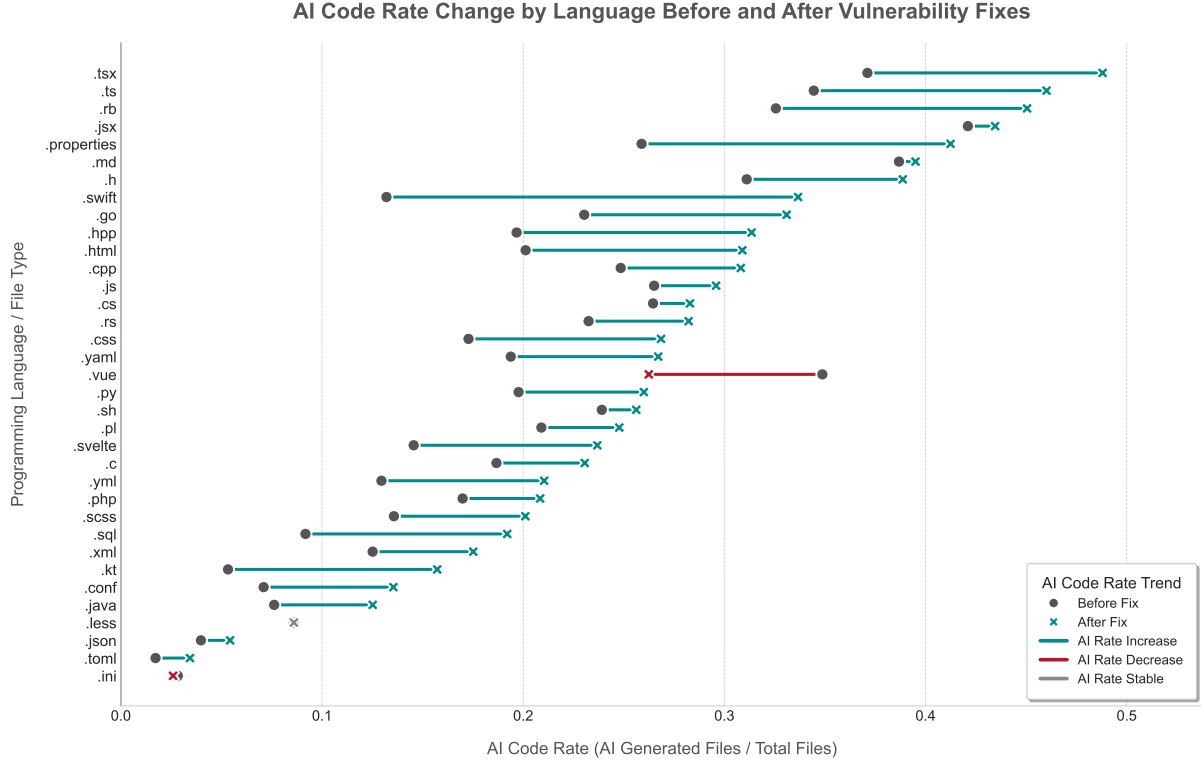


Figure 4: Distribution of AI Code Ratios in Different Programming Languages Before and After Vulnerability Remediation

However, this growth is not homogeneous. There are significant differences in the growth amplitude of AI code ratios among different types of programming languages and files, which reflects the penetration depth and application patterns of AI technology in different development paradigms. For example, in some languages widely used for building dynamic applications or containing a large amount of boilerplate code, the growth of AI code ratios is particularly obvious, which may indicate that AI tools provide more direct and efficient assistance in these fields. Such differentiated penetration may also be related to factors such as the volume of training data of AI models for specific languages, the structural complexity of the languages themselves, and the adoption level of AI tools by the community.

Although the AI code ratio has generally increased, it is worth noting that in a small number of specific programming languages or configuration file types, the AI code ratio remained stable or even experienced a decline after vulnerability remediation. This downward trend provides a crucial security signal: it indicates that in these types of languages, AI-generated code may be directly identified as contributors to vulnerabilities. When these defects are fixed, the relevant AI code segments are removed or replaced accordingly, and their proportion in the overall codebase decreases.

The general increase in AI code ratios of different programming languages after vulnerability remediation reflects the widespread application of AI tools in development and maintenance. However, the decline in AI code ratios of a small number of languages

clearly indicates that AI-generated code may directly lead to vulnerabilities under specific programming paradigms, emphasizing that the security assessment of AI-generated code needs to be carried out at the language level to address its differentiated potential risks.

4.3 Summary

This chapter conducts an in-depth review of the current penetration status of AI code generation technology in the open-source ecosystem from two dimensions: the temporal evolution trajectory and the distribution of programming languages.

Data reveals that the industry's application of AI has moved beyond the frenzy period of blind adoption, entering a new stage of rational symbiosis between humans and AI, and establishing a collaboration paradigm of "AI for efficient output, humans for critical oversight". However, the structural decline in the AI code ratio of specific programming languages after vulnerability remediation objectively confirms the risk attribute of AI-generated code as a "source of defects" in specific scenarios.

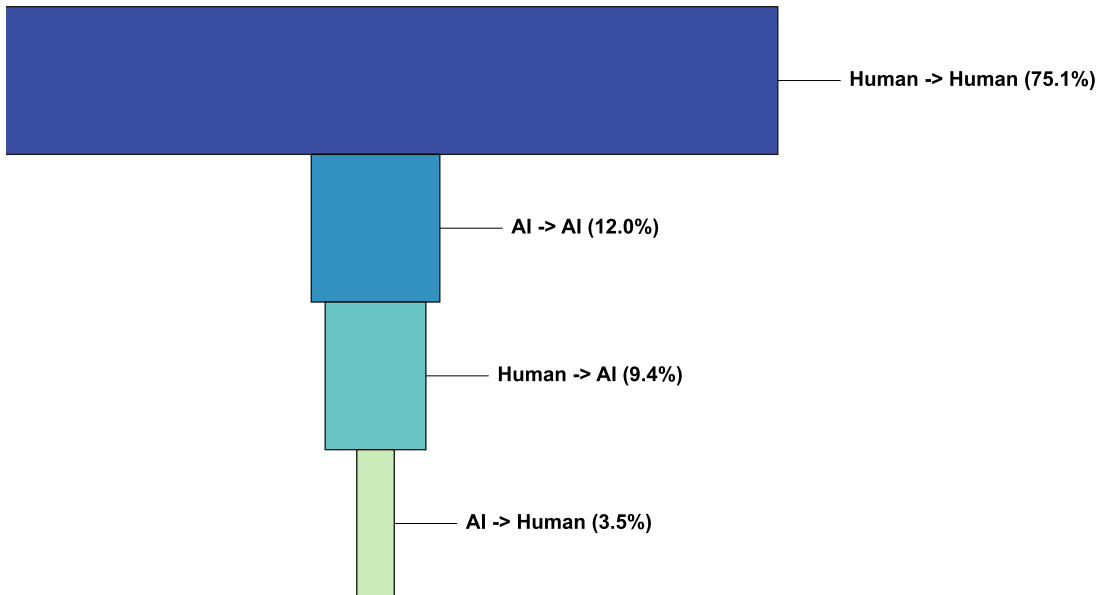
This complex situation, where efficiency dividends coexist with potential hidden dangers, indicates that automation relying solely on tools has reached a ceiling. Software engineering is transforming into a hybrid development model that demands higher discrimination capabilities and systematic risk control.

5 Security Risks and Vulnerability Characteristics of AI-Generated Code

5.1 Evolution of AI's Role in the Vulnerability Lifecycle

AI code generation technology is exerting a significant impact on the generation, remediation, and evolution processes of vulnerabilities. This section, from the perspective of identified Common Vulnerabilities and Exposures (CVE), will conduct an in-depth analysis of the dynamic changes of AI-generated code before and after vulnerability remediation, explore the complex role of AI-generated code as a potential source of vulnerabilities or a remediation tool, and reveal the detailed correlation between AI-generated code and the security posture of projects.

Macro-Analysis of AI Code Transformation in Vulnerability Fixes



Total Transformations: 1,766

Figure 5: Changes in AI and Manual Conditions Before and After Vulnerability Remediation

After analyzing a large volume of CVE data (Figure 5), a key phenomenon was observed: in the remediation process of some vulnerabilities, the code originally generated by AI was significantly replaced with manually written code. This "AI-to-human" pattern is not an isolated incident, but recurs in multiple CVE scenarios. This indicates that in these specific vulnerabilities, AI-generated code itself may directly serve as the root cause of security defects. When vulnerabilities are discovered and addressed, security engineers

and developers realize that the existing AI-generated code has problems, and choose to replace it with solutions that have undergone manual review and verification. This finding provides concrete evidence for the risks mentioned earlier in this report that AI-generated code may introduce Code Hallucination or inherit insecure patterns in training data. It compels us to examine the intrinsic quality and security of AI-generated code, especially the potential hidden risks that may arise when it is integrated into critical business logic on a large scale.

In contrast to the above phenomenon, another notable trend indicates that AI-generated code also plays an active auxiliary and restructuring role in the vulnerability remediation process. In many CVE remediation cases, a large amount of code originally written manually was converted to AI-generated code after remediation. This "human-to-AI" pattern reflects that when addressing vulnerabilities, developers are increasingly using AI tools to accelerate code restructuring, generate new security features, or leverage them to implement more efficient patch application. This demonstrates the great potential of AI in rapid iteration and response to security incidents.

However, this trend also raises new considerations: large-scale restructuring of existing code through AI tools, even to fix known vulnerabilities, may inadvertently introduce new complexity. When the AI model itself fails to fully understand the security context or best practices, it may even generate new, more difficult-to-track defects. Therefore, the efficiency advantages of AI in remediation must be paired with strict verification mechanisms to ensure the robustness of the remediation solutions.

Examining the evolution of AI-generated code from the CVE perspective, AI-generated code plays a dual role in the vulnerability lifecycle: it may be manually replaced as a direct source of vulnerabilities, and is also widely used to assist and accelerate the vulnerability remediation process. This dynamic interaction reveals that the quality and security of AI-generated code are not static, and the efficient introduction of such code implies a continuous need for review and verification. Future security strategies must fully integrate the characteristics of AI-generated code and establish a full-lifecycle security governance framework from generation to maintenance, in order to address the complex challenges brought by AI-driven development.

5.2 Characteristic Profile of AI-Introduced Vulnerabilities

Unlike vulnerabilities caused by human developers due to negligence in complex business logic or deviations in requirement understanding, the defects introduced by AI-generated code exhibit a strong patterned characteristic. This characteristic essentially stems from the large language model's mechanical imitation of historical coding patterns in training data, rather than a deep understanding of security principles.

Our empirical data (Figure 6) shows that vulnerabilities introduced by AI are not evenly distributed across all types, but present a highly concentrated trend, mainly focusing on two areas: Input Validation and Data Encoding and Unsafe API Calls: 1. First, AI shows obvious shortcomings when dealing with scenarios involving contextual security boundaries. For example, when performing data serialization, formatted output, or processing external input streams, AI often struggles to automatically implement correct escaping or cleansing strategies based on the current contextual environment. This lack of awareness of security context makes injection-type risks one of the most common hidden dangers in AI-generated code. 2. Second, AI tends to reuse outdated practices in training data. Since the model learns from massive amounts of historical code, it is prone

to calling outdated algorithms or hash functions that have been deprecated by modern security standards when it comes to cryptography or sensitive operations. This "copying the existing pattern" generation logic of AI results in the code it generates being syntactically flawless, but often lagging behind current standards in terms of security best practices.

It is worth noting that vulnerabilities introduced by AI rarely involve deep-seated business logic errors. Instead, they are more reflected as local implementation flaws at the code snippet level. This "shallow but high-frequency" distribution characteristic indicates that although AI introduces new risk sources, these risks can to a large extent be effectively identified and intercepted through mature static analysis tools and automated scanning rules.

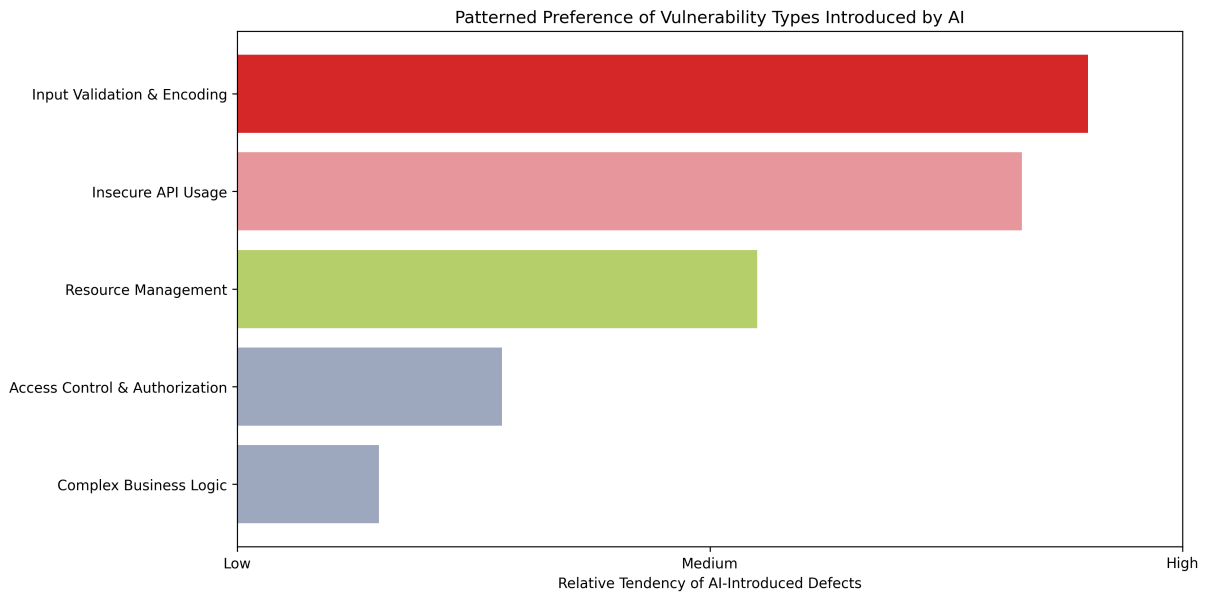


Figure 6: Pattern Analysis of Types of Vulnerabilities Introduced by AI

Further empirical research on the risk characteristics of vulnerabilities introduced by AI (Figure 7) shows that the risk characteristics of vulnerabilities introduced by AI are not "low-level", but present a dual trend of Severity Anthropomorphism and Attack Surface Networking. 1. First, in the dimension of vulnerability severity, AI-generated code shows an astonishing consistency with human developers. Statistical distribution shows that the severity level of vulnerabilities introduced by AI is not significantly lower than that of humans, and it faithfully "inherits" the risk distribution pattern of human developers. This means that AI not only makes mistakes, but also fully has the ability to introduce high-risk vulnerabilities that lead to serious consequences such as system paralysis and data leakage. AI-generated code does not have a natural "immunity" or "downgrade effect" in terms of security, and its upper risk limit is completely equal to that of human-written code. 2. Second, in the dimension of attack vectors, vulnerabilities introduced by AI show a more dangerous structural preference – a networking tendency. Unlike vulnerabilities that can only be triggered mainly by relying on local access or physical contact, defects in AI-generated code are more significantly concentrated on network attack vectors. This trend is particularly prominent in code scenarios that handle API interface calls, Web service interactions, and network protocol parsing.

This distribution characteristic means that code modules generated with AI assistance

are often more likely to become breakthrough points for remote attackers. Attackers do not need to obtain local system permissions to trigger these vulnerabilities by sending malicious requests via the Internet. Therefore, the widespread application of AI has invisibly expanded the remote attack surface of software systems, putting greater pressure on the boundary defense of the systems. This warns us that when reviewing AI-generated code involving network communication and external interfaces, we must raise the security baseline standards, rather than merely relying on the correctness of its function implementation.



Figure 7: Analysis of Risk Characteristics of Vulnerabilities Introduced by AI

5.3 Summary

This chapter focuses on the Vulnerability Full Lifecycle and systematically analyzes the security essence of AI as a double-edged sword. Empirical research shows that AI can not only act as an auxiliary tool to accelerate vulnerability remediation, but also, due to its mechanical imitation characteristic, directly become a source of high-risk vulnerabilities such as Missing Input Validation and Unsafe API Calls.

The defects introduced by AI show prominent characteristics of Severity Anthropomorphism and Attack Surface Networking: the harm level of vulnerabilities generated by AI is no less than that of those created by humans, and they are more inclined to expose the Remote Network Attack Surface.

This finding breaks the stereotype that AI only produces low-level syntax errors, and warns us that while enjoying the development convenience brought by AI, we must face up to its inherent shortcomings in deep business logic and contextual security awareness, and establish targeted defense mechanisms.

6 Risk Mitigation Recommendations

In the face of the inherent endogenous defects of AI code generation technology and the complex supply chain security challenges it triggers, relying solely on defense measures in a single dimension has become insufficient. This chapter proposes the construction of a comprehensive and multi-dimensional security defense system. As shown in Figure 8, coordinated efforts are made from three dimensions: Evaluation Benchmarks, Model Intrinsic Security, and Human-Machine Collaborative Governance, aiming to build a Full-Lifecycle Security Closed Loop from code development to deployment.

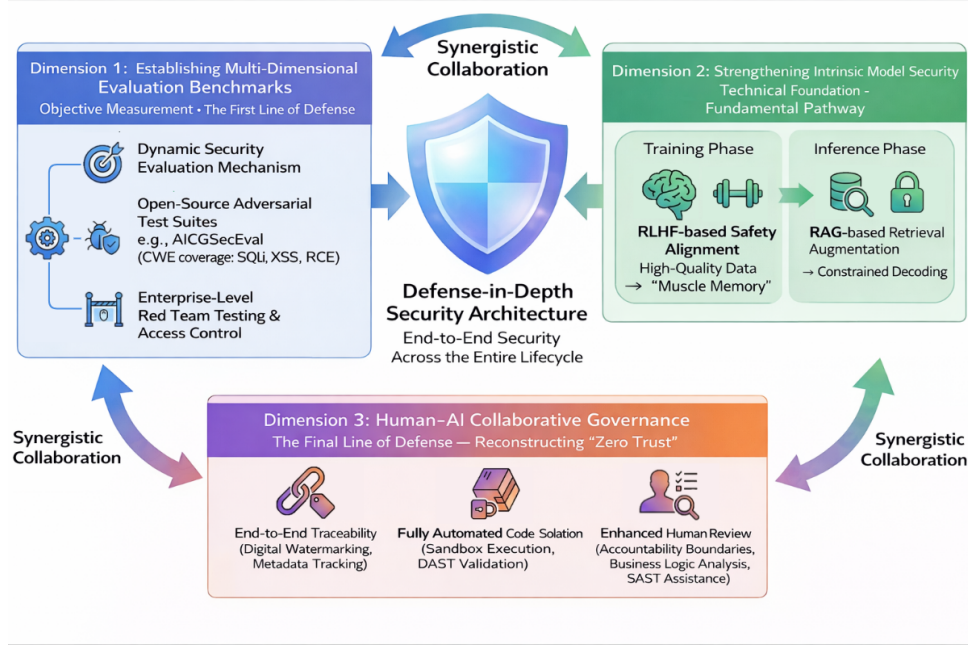


Figure 8: Risk Mitigation Framework

6.1 Establish Multi-dimensional Evaluation Benchmarks

The probabilistic and random characteristics of AI model outputs make traditional static testing methods difficult to effectively cover their security boundaries. Therefore, it is necessary to establish a dynamic security evaluation mechanism specially for large language models, which serves as the first line of defense before model access and launch. There are already relatively mature open-source practices in the industry, such as the **A.S.E** project. These tools build adversarial test sets covering high-risk scenarios such as SQL injection, Cross-Site Scripting (XSS), and remote code execution based on the Common Weakness Enumeration (CWE), which can quantitatively evaluate the generation security of models in multi-language and multi-scenario environments. Enterprises should establish a "Red Team Testing" and access mechanism based on such mature benchmarks at the model introduction stage. Through strict baseline testing and continuous attack and defense drills, it is ensured that the online models have sufficient robustness, reducing the application of models with native security defects from the source.

6.2 Enhance Model Intrinsic Security

Improving the intrinsic security of the model is the fundamental way to solve the problems of Code Hallucination, logical vulnerabilities, and Knowledge Truncation, which needs to start from two key stages: training alignment and reasoning enhancement. Training Stage: Emphasize source governance, introduce high-quality datasets containing secure coding specifications and conduct strict data cleaning. Use Reinforcement Learning from Human Feedback (RLHF) technology for in-depth security alignment, and guide the model to internalize secure programming paradigms through the mechanism of penalizing unsafe outputs and rewarding compliant code, so that the model forms "muscle memory" for secure coding like human senior experts. Reasoning Stage: To address the problem of knowledge lag in model training data, it is recommended to adopt Retrieval-Augmented Generation (RAG) technology to dynamically mount the latest CVE vulnerability database and authoritative security announcements as external knowledge sources, so that the model has real-time risk avoidance ability and context awareness ability in the code generation process. At the same time, Constrained Decoding technology can be combined to forcibly constrain the output structure of the model from the Abstract Syntax Tree (AST) level, blocking the calling of non-secure APIs and the generation of dangerous code patterns from the underlying logic, and forcing the model to output code that strictly complies with syntax and security specifications.

6.3 Human-Machine Collaborative Governance

In view of the fact that current AI technology cannot achieve complete zero false positives and zero risks, it is necessary to reconstruct the Zero Trust mechanism in the software supply chain process and strengthen the key role of manual review in the security closed loop. It is recommended to implement full-process traceable management, conduct mandatory digital watermark marking or metadata tracking on AI-generated code; for fully automatically generated code (such as output from AI Agent), strictly implement sandbox isolation operation and Dynamic Application Security Testing (DAST) verification by default. At the governance level, it is necessary to reshape the development specifications adapted to the era of AI-assisted programming, clarify the power and responsibility boundary of "developers are the ultimate responsible persons for code security", and prevent responsibility transfer caused by over-reliance on tools. The focus of code review should shift from traditional syntax error checking to deeper business logic consistency verification and data flow security analysis. Use Static Application Security Testing (SAST) tools to assist in identifying hidden defects, ensuring that in the human-machine collaboration mode, human engineers always have the final decision-making power and control over the software security status.

6.4 Summary

Risk governance of AI code generation is a systematic project. Establishing evaluation benchmarks provides an objective measurement, enhancing the intrinsic security of the model consolidates the technical foundation, and human-machine collaborative governance builds the final security defense line. The three complement each other, jointly forming a Defense in Depth security system. Only through the deep integration of technical means and management processes can we effectively curb the potential security risks

while releasing AI productivity, and promote the evolution of software engineering in a more efficient and secure direction.

7 Conclusion

With the in-depth application of Large Language Models (LLMs), AI code generation technology has significantly improved software development efficiency, while also reshaping the security boundaries of the software supply chain. As an emerging "code contributor" in the software supply chain, AI's inherent characteristics of Code Hallucination and Knowledge Cutoff can directly lead to code with logical defects or known vulnerabilities (CVEs) being injected into projects. In addition, the risk of sensitive data leakage during interaction with AI services, as well as the ambiguity of the source of generated code, constitute severe compliance and intellectual property challenges. Empirical analysis further reveals that AI plays a dual role in the vulnerability lifecycle: it is both an efficiency tool, widely used to accelerate vulnerability remediation and code restructuring; however, in some scenarios, AI-generated code itself is the direct source of vulnerabilities, and is replaced by manual code during the remediation process. This duality highlights the long-term risks that over-reliance on AI may erode the security culture of development teams, leading to Automation Bias and core security Skill Degradation. Therefore, integrating AI into the development process safely requires the establishment of a comprehensive governance framework that goes beyond traditional code audits, transforming developers from pure "creators" into "validators" with highly critical review capabilities for the output of human-machine collaboration. In response to the above challenges, this report proposes to build a three-in-one defense-in-depth system of "Evaluation Benchmarks - Model Security - Collaborative Governance". This requires establishing a dynamic security evaluation and red team testing mechanism at the introduction stage; improving intrinsic security through RLHF alignment and RAG retrieval enhancement at the model level; reconstructing a "Zero Trust" human-machine collaboration mode at the application level, and implementing code traceability and sandbox isolation. Through the deep integration of technical enhancement and management reshaping, developers are transformed from pure "creators" into highly critical "validators", ensuring the resilience and security of the software supply chain in the intelligent era.